Pendulum Oscillation Meter

Mechatronics Engineering Project – Salvador Martínez Santoyo

1. Introduction and Motivation

I was inspired by automatic watches: how they work, their history, the precision they must have, and the perfect engineering they represent. As a mechatronics engineering student, I want to do semester projects related to watchmaking because it's what I'm passionate about and because I know they will bring me closer to the companies I want to work for in the future.
I chose a pendulum because the oscillator is one of the most important parts of a watch. It's what defines whether it keeps time correctly or not. I wanted to start there, understanding how an oscillation is measured and how deviations are detected.
With this project I was able to apply knowledge of microcontroller programming, creativity, and problem-solving. It was a 100% improvised project, not copied from anyone; it came from a brainstorming session. This is the first in a series of projects I plan to do every semester.

2. Objectives

Main objective:
Measure the frequency of a pendulum, detect deviations, and simulate how a company like NOMOS could approve or reject a balance wheel.
Secondary objectives:

- Develop my creativity and improvisation.
- Practice with Arduino and microcontroller programming.
- Learn to use new sensors.
- Research oscillations: I dedicated two weeks at the beginning to studying formulas and concepts, which helped me better understand how watches work.

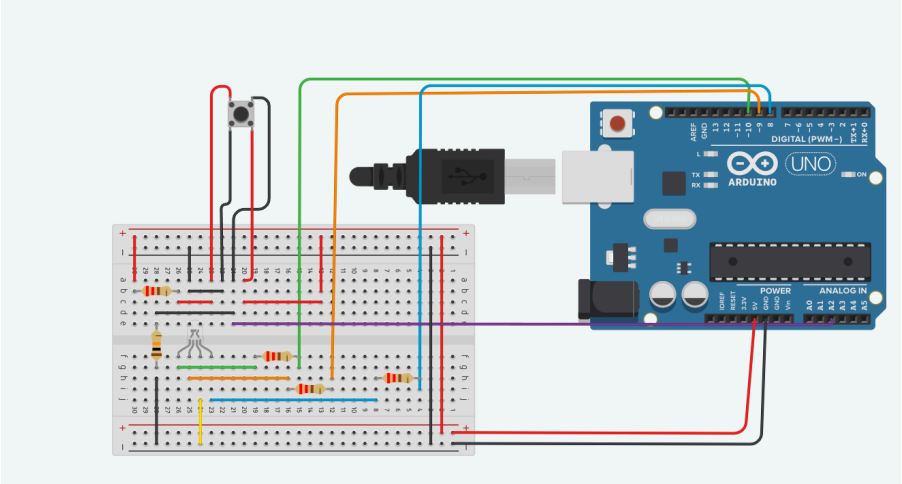
3. Components and Materials

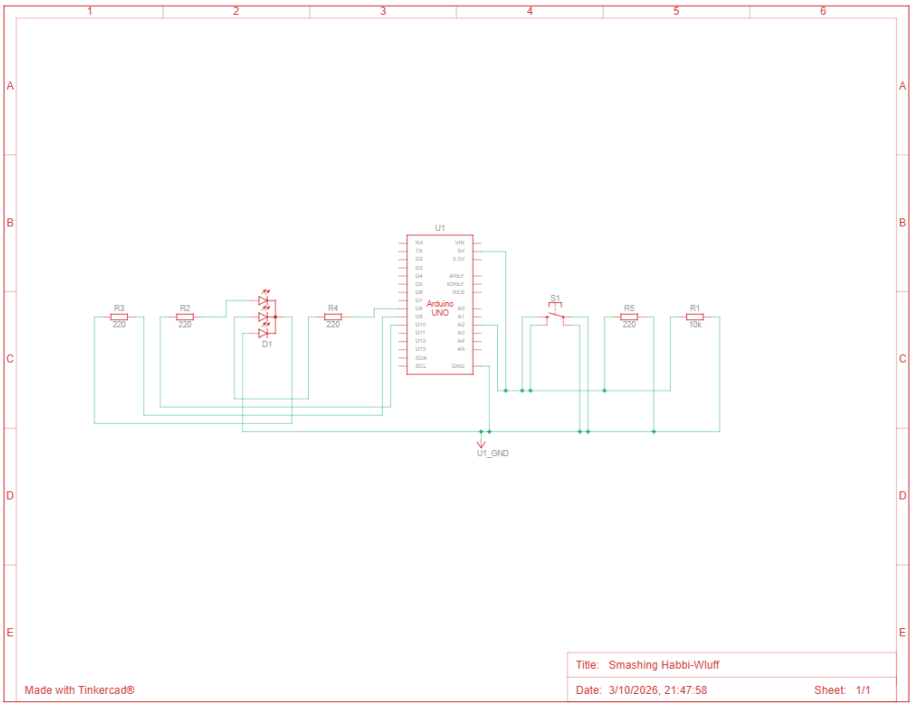
- Arduino
- CNY70 sensor
- RGB LEDs
- Wires
- Breadboard
- Soldering iron and solder (for soldering)
- 3D link: https://www.tinkercad.com/things/8wTF3N63Cec-arduino/edit?returnTo=https%3A%2F%2Fwww.tinkercad.com%2Fdashboard&sharecode=64VfJK0amhxsJZazQRpXr72tS7N-ugr_MZBMteFRU4E

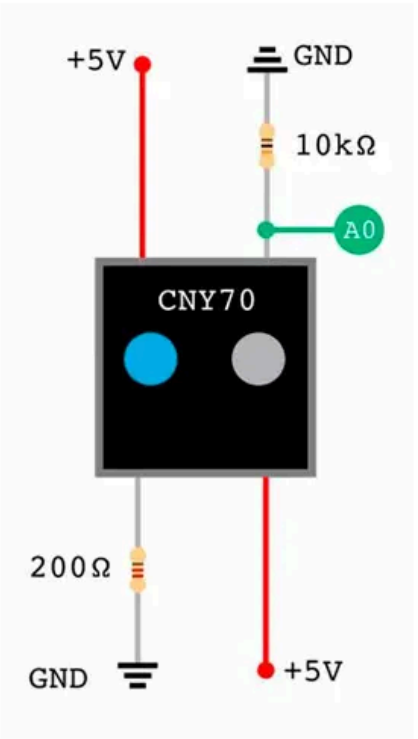
Note about the simulation:

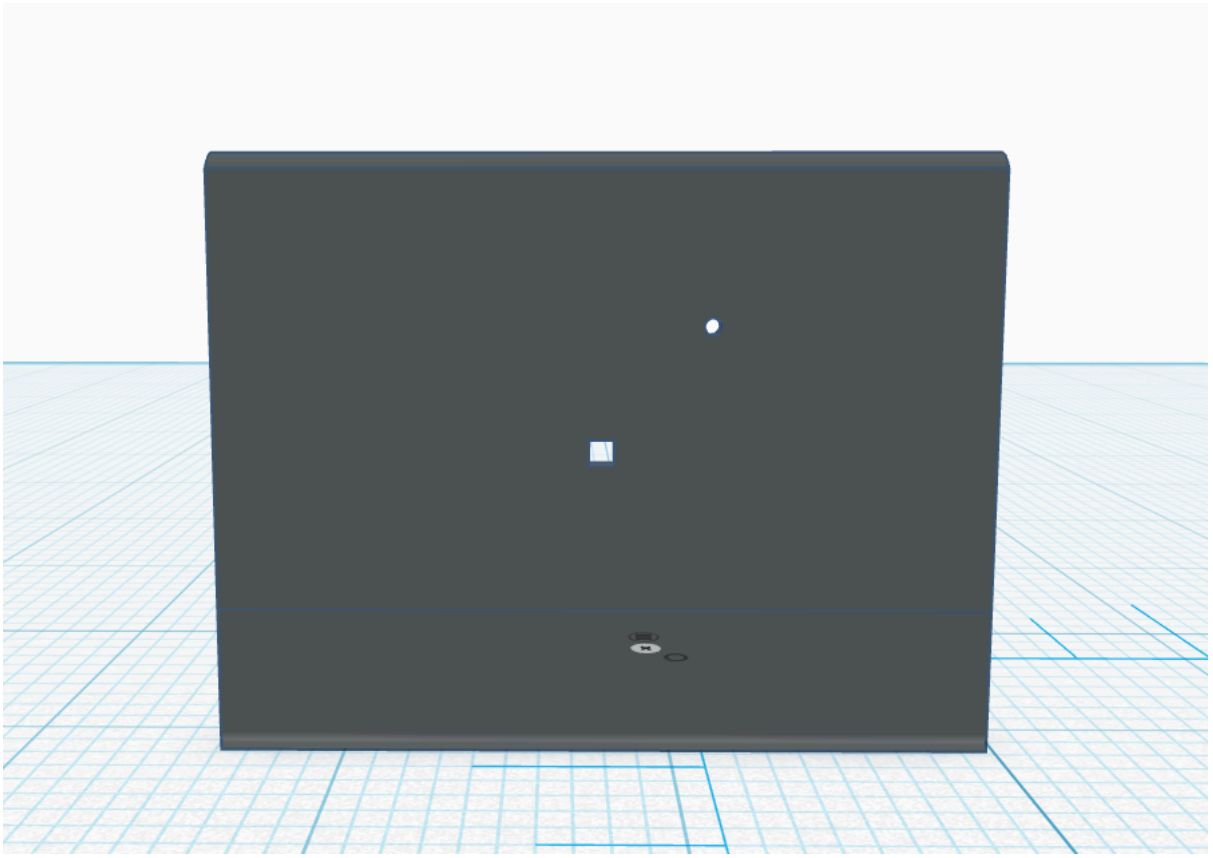
For the circuit design, I created a schematic and a digital simulation using Tinkercad. Since the CNY70 sensor is not available on that platform, I used a push button as a replacement. It's important to emphasize that the position of the button in the image is not arbitrary: it is placed exactly where the sensor should be connected. The wires seen in the simulation (on the left) show how the real sensor should be connected. In other words, the button is positioned exactly as the CNY70 should go, and the connections seen in the image are the same ones used with the physical sensor. In the image on the right, you can see that the 200-ohm resistor is on the negative wire, but in my case I connected it on the positive.

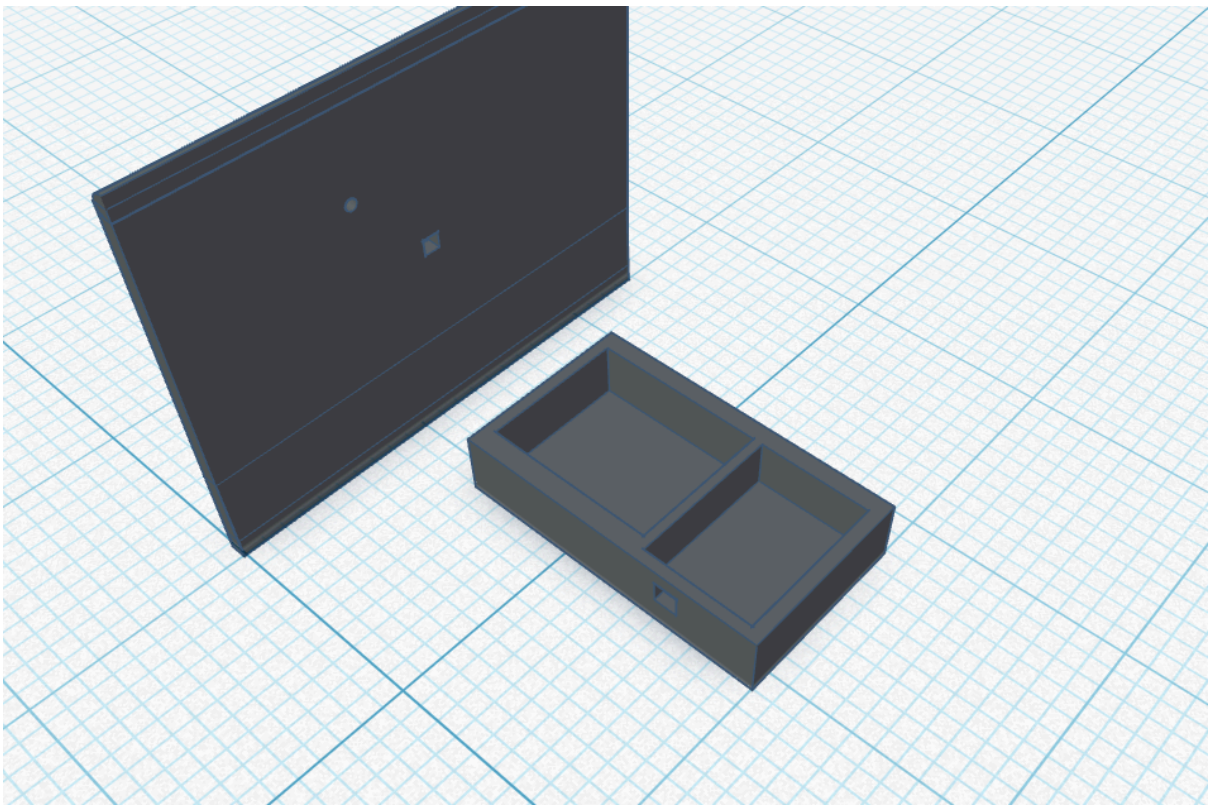
-











4. How It Works (Theory)

The system measures the oscillation period of the pendulum using a CNY70 reflective infrared sensor. Every time the pendulum passes the point where the sensor is, it detects it.

The Arduino measures the time between two consecutive passes of the pendulum (one complete period) using the micros() function. With that time, the frequency is calculated (f = 1/T). If the frequency remains stable within an acceptable range, the pendulum is considered "valid." If there is too much variation, it is considered "defective."

5. Construction and Assembly

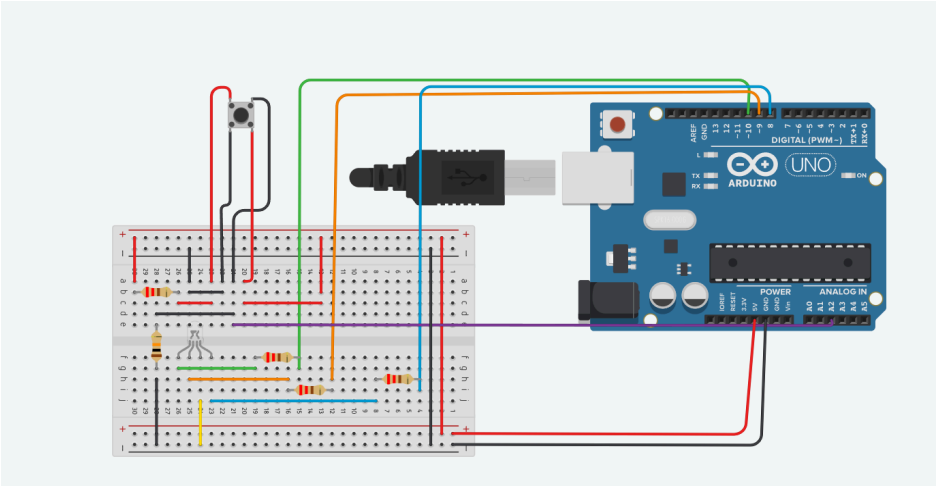
The pendulum is a simple purchased pendulum, consisting of a rod and a string. I did not focus too much on the construction of the pendulum itself, but rather on the measurement system.

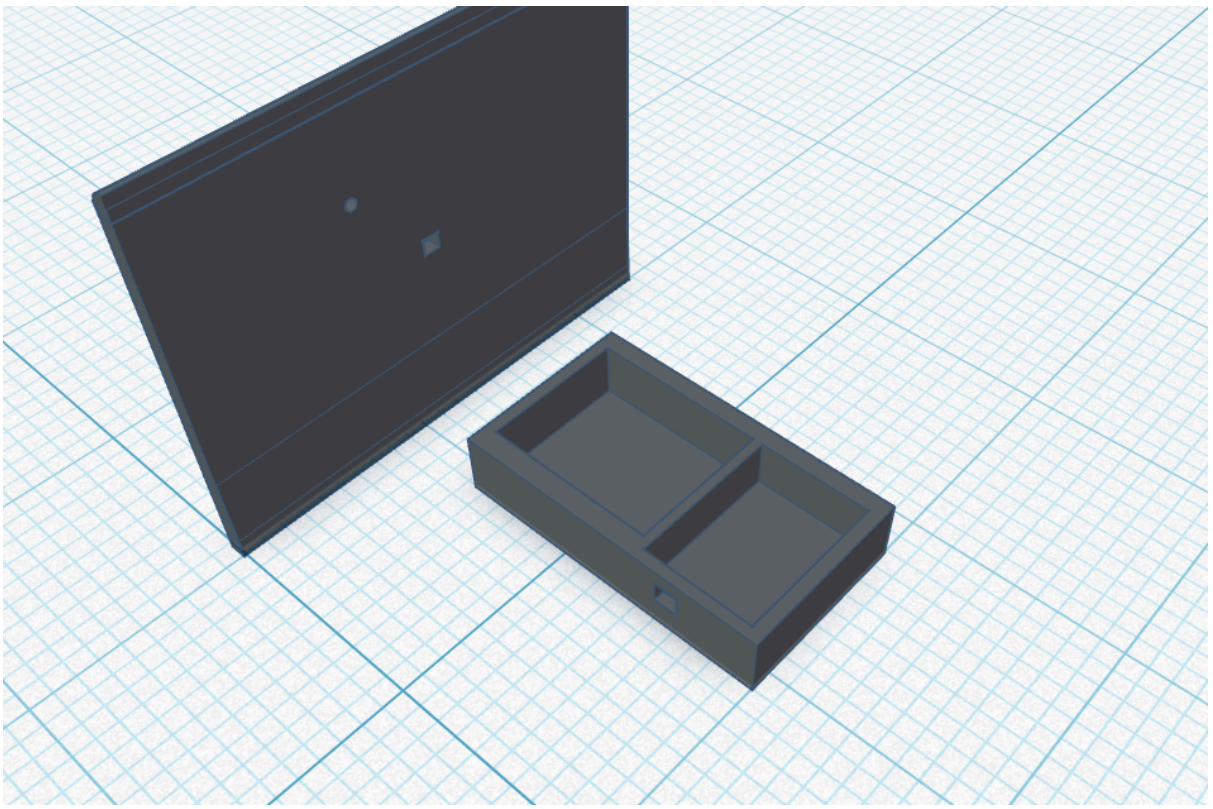
I designed the base in Tinkercad and had it 3D-printed at school. I took the measurements for all the components myself, created the design, and managed the print order at the university. The mechanical part of the project, including the base for the Arduino and the breadboard, the holder for the sensor and the LED, as well as the support for the pendulum, was entirely designed and assembled by me. The CNY70 sensor was embedded exactly where the pendulum passes, a few millimeters away, with a calibration in the programming so that it detects it correctly on every swing.

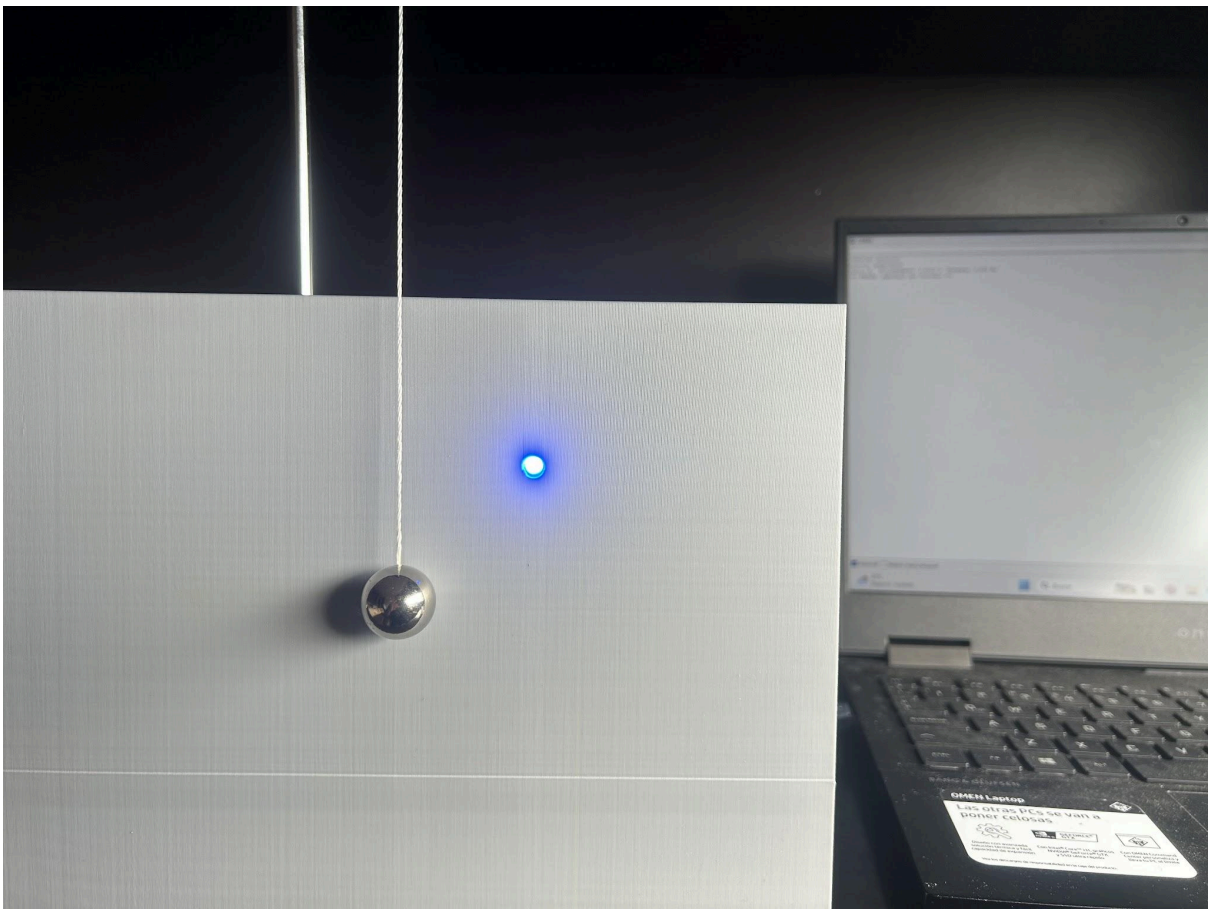
The RGB LED was placed slightly above and to the right of the sensor to visually indicate the status of the system. I soldered the sensor and the LED: one part was soldered to the sensor so it would sit firmly, and the other part was connected with a regular cable to the breadboard.

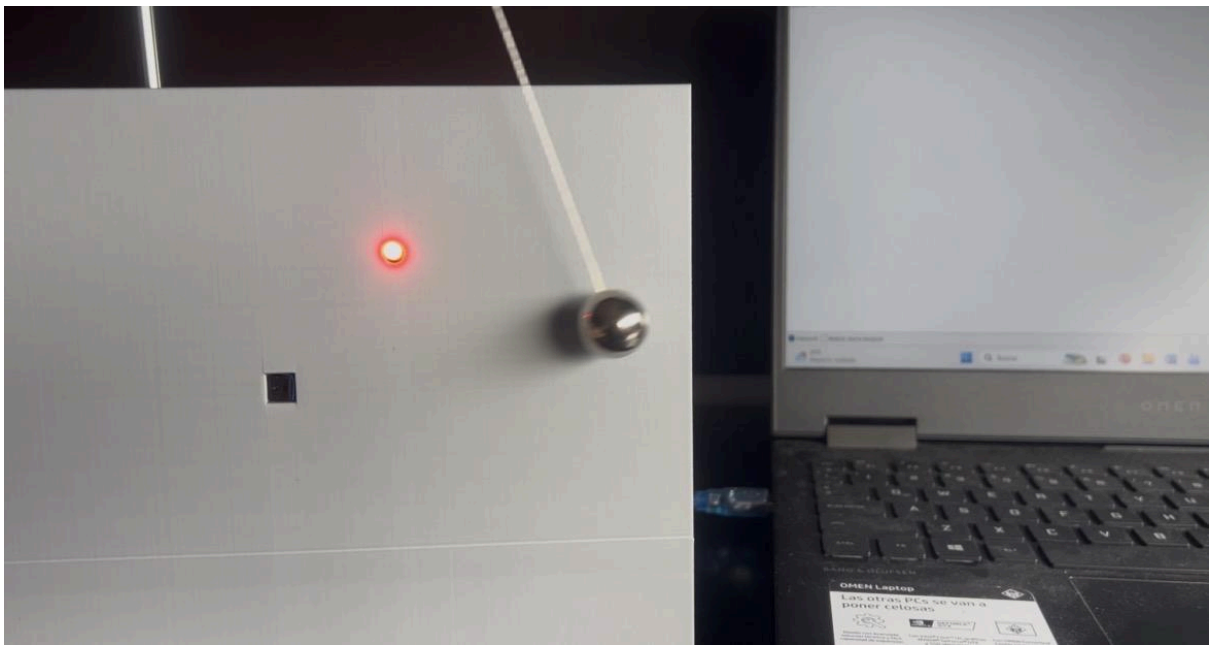
The Arduino and the breadboard are loose, not glued. I left them like this because I find it more practical: if I need to change the oscillator or make adjustments, I can move them without any problem. They are in a place where they don't get in the way and can be easily rearranged.

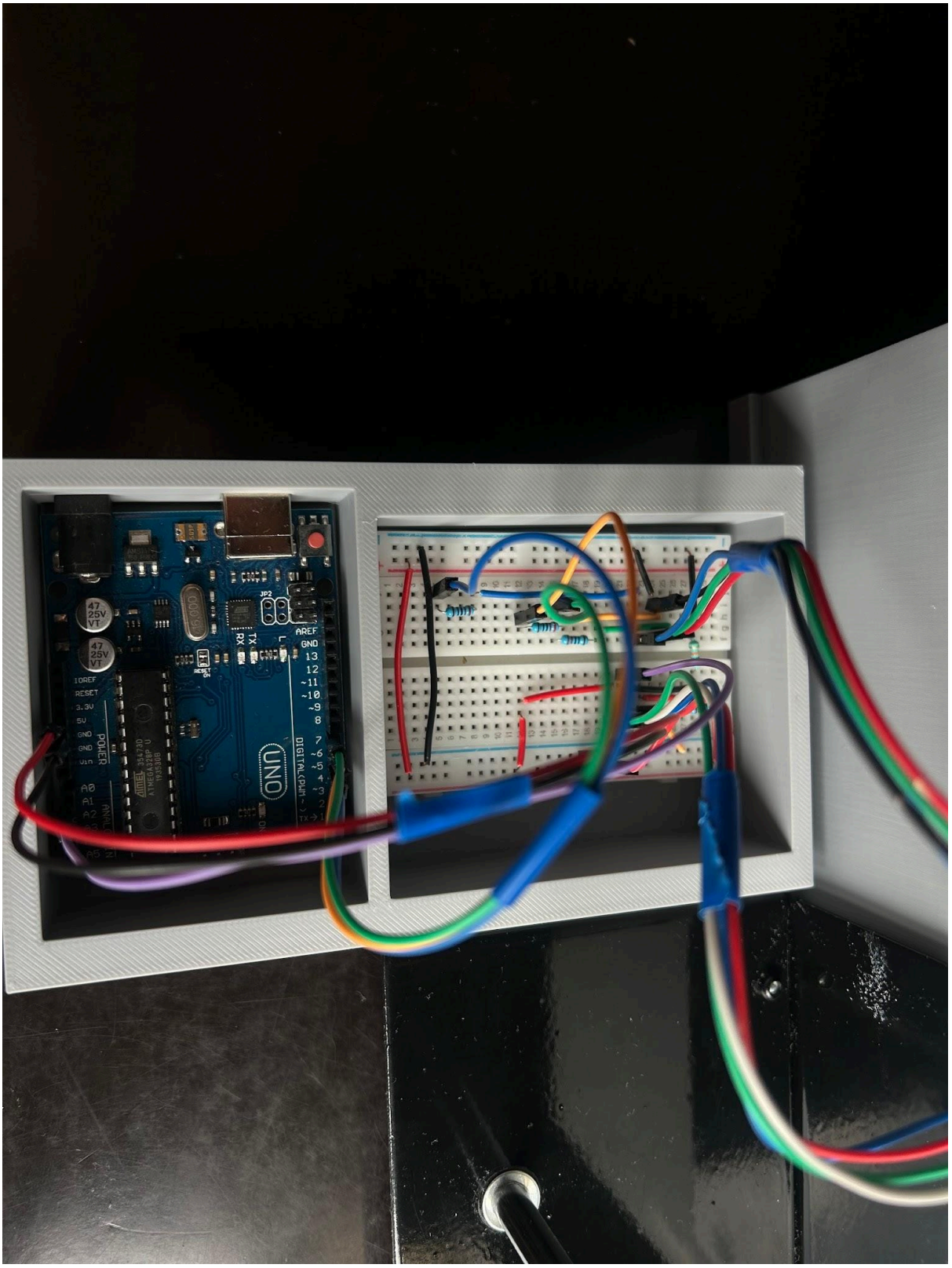
The pendulum swings freely. I only did some calculations to determine the distance and the oscillation time.

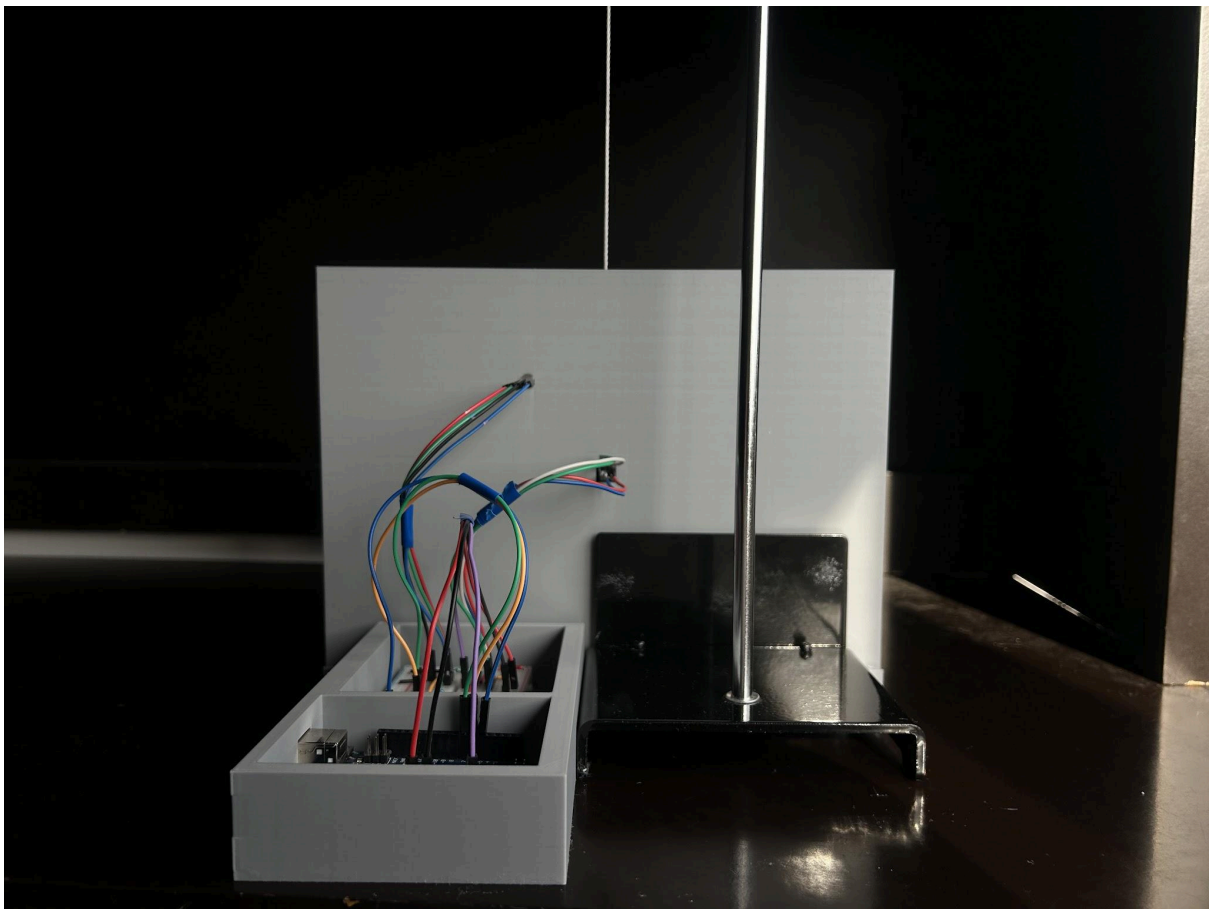


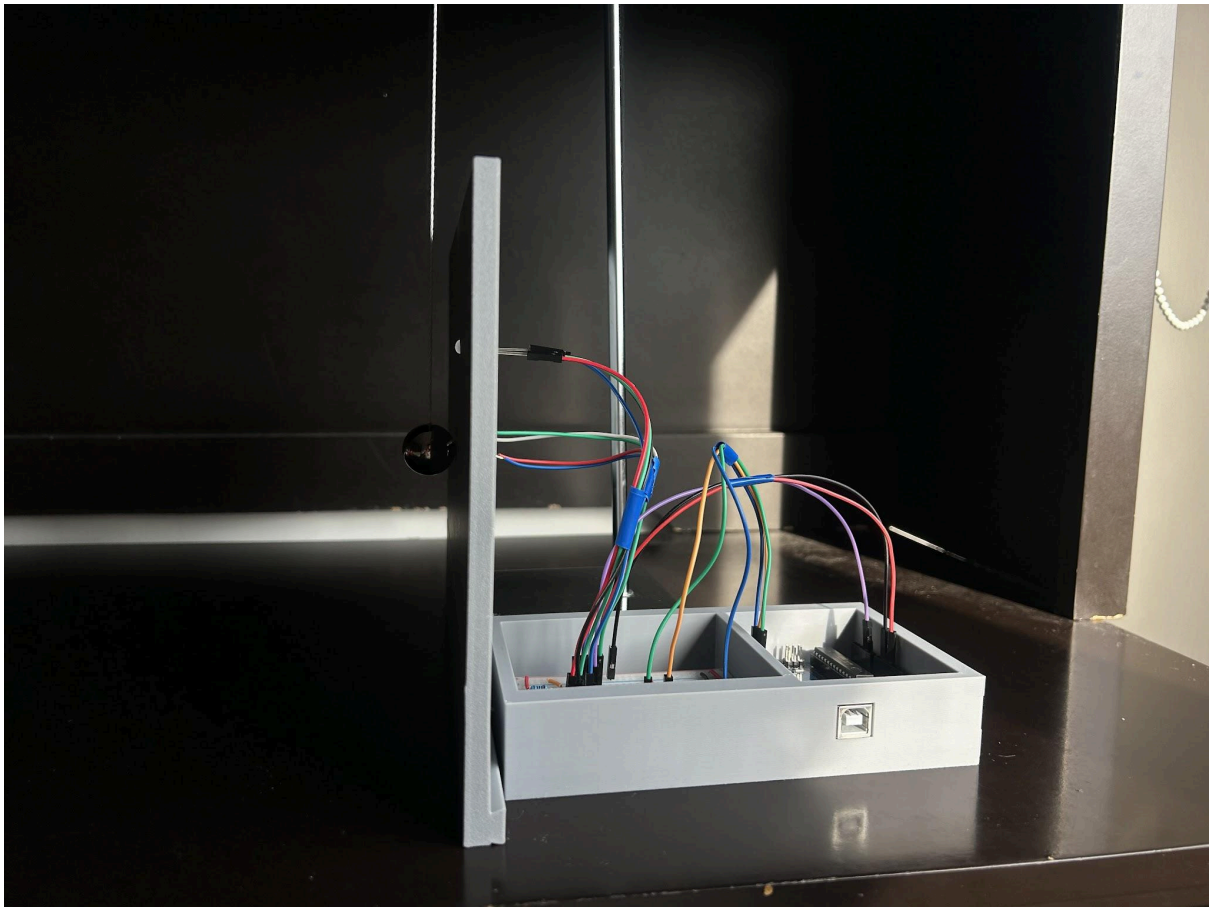



OMEN Laptop
Las otras PCs se van a poner celosas


OMEN Laptop
Las otras PCs se van a poner celosas


47
25V
VT
47
25V
VT
JP2
AREF
GND
13
12
~11
~10
~9
8
7
~6
~5
4
~3
2
TX→1
RX←0
IOREF
RESET
3.3V
5V
GND
GND
Vin
POWER
A0
A1
A2
ANALOG
UNO
DIGITAL(PWM~)





6. Programming

The structure of the code was one of the biggest challenges of the project. I had to design a logic that would count each detection from the sensor and distinguish between an odd detection (when the pendulum passes for the first time) and an even one (when it returns). Only when the pendulum completes a full cycle (back and forth) is one complete period counted.
The program uses micros() to measure the time between detections with high precision. When the sensor detects the pendulum's pass, the Arduino saves the current time. When the next odd detection occurs (which completes the period), it calculates the difference between both times and obtains the period (T). With that value, it calculates the frequency using the formula f = 1/T.
To avoid bouncing or false detections, the code includes edge detection (only counts when the sensor goes from not detected to detected) and a small 2 ms delay as debounce.
The RGB LEDs indicate the system's status:

- Red: waiting for detection
- Green: successful detection (blink) / successful program completion
- Blue: error (frequency out of tolerance)

The program also sends the data to the serial monitor, where you can see the cycle number, the period in seconds, and the frequency in Hz.
The code is organized in blocks: pin configuration, sensor reading, time and frequency calculation, and visual output. It includes comments to make it easy to understand and modify.
The system has a tolerance range of 0.75 to 1 Hz. If the measured frequency falls outside that range, the blue LED turns on and the program stops, indicating that the oscillator is defective. If 5 cycles are completed within the range, the program indicates that the oscillator passed the test and stops.

7. Results

The average frequency obtained was approximately 0.85 Hz, with a variation between 0.8 and 0.9 Hz. These values match the theoretical calculations I made before assembling the system.
The program has a tolerance range of 0.75 to 1 Hz, which equals a margin of approximately 10-15% over the expected frequency. This margin is configurable and can be adjusted according to the needs of whoever uses the system.
In general, when the sensor correctly detects each pass of the pendulum, the variation is minimal. The problem occurs when the sensor doesn't detect a pass properly (it skips a detection), which can happen occasionally. This generates an erroneous reading, but it's a detection problem, not a problem with the program's logic.
Video
https://youtu.be/fMC7iJrH5pU?si=q8NbWx_Mj7CZ83-r

8. Problems and Solutions

During the development of the project, I encountered several challenges. Instead of seeing them as failures, I used them as opportunities to iterate and improve my engineering process:

The project base (3D Printing Iteration). The challenge was to design the base in Tinkercad so that all components (Arduino, breadboard, sensor, LED, and pendulum) would fit perfectly. My first 3D print attempt failed because the tolerances were not correct. I had to re-measure the components, adjust the 3D model, and reprint it. Lesson learned: The importance of iterative design and precise tolerance management.

The programming logic. Designing the logic to distinguish between odd and even detections to calculate a full period took me several days. I had to build it from scratch, test it, and debug it step by step. Lesson learned: Systematic debugging is key to complex logic.

The use of millis() and micros(). I initially lacked the knowledge to implement these functions for precise time measurement. Overcoming this learning curve was challenging, but once understood, it became a highly reliable tool. Lesson learned: Time management in microcontrollers requires non-blocking code.

The CNY70 sensor (Hardware limitation). The sensor occasionally missed the pendulum's pass due to false triggers. I solved this by implementing edge detection in the software, calibrating the threshold, and adding a 2ms delay to debounce the signal. Lesson learned: Software can compensate for hardware imperfections.

Working alone (Project Management). I handled every aspect: planning, research, 3D design, purchasing, assembly, and troubleshooting. Combining this with my university schedule was demanding, but I successfully delivered the project in 6 weeks. Lesson learned: Self-discipline and time management are as crucial as technical skills.

9. Conclusions

With this project, I learned a lot about oscillations and was able to dive deeper into how balance wheels work. I also remembered several things I already knew, like soldering, and I learned programming methods I didn't know, like using millis() and micros() for time measurement. That was a big challenge, but very useful.

It was quite an exhausting project because of everything I mentioned: I did it all alone, from the idea to the construction. Combining it with school was tough, but I managed to finish it in 6 weeks, even though it was planned for the whole semester.

If I were to do it again, I would further optimize the mechanical details. The mechanical part, especially the design of the 3D-printed base, was a great learning process. I took the measurements and designed it all myself in Tinkercad. For a future version, I might design even more complex structures, but the current 3D-printed base gives the project a very professional look.

To close, I want to say that in the video it looks simple, like something light. It is something that cost me a lot of time and effort because I made it from scratch, without references or manuals, just with ingenuity.

10. Acknowledgments

Dear NOMOS team, dear person reading this:
Truly, thank you very much. If you made it this far and dedicated your time to reading everything I wrote, to reviewing this small project that I did alone, you don't know how much it means to me.
I know your time is valuable and that you have a thousand more important things to do. That's why the simple fact that you take a glance at this is already a dream for me. If you can also give me feedback, however small, I would appreciate it so much. You don't know how much it means to me.
This project was work, yes, but it was also passion. And that you, who make the watches I admire so much, lend me your time and attention, is something I will not forget.
Thank you very much, truly.
— Salvador Martínez Santoyo

Referencias

Arduino. (s.f.). micros(). Arduino Documentation. Recuperado el 4 de octubre de 2026, de https://docs.arduino.cc/language-reference/en/functions/time/micros/
Ferrovial. (2023, 9 de enero). Oscilaciones y ondas: qué son, relación entre ambas, tipos e importancia. https://www.ferrovial.com/es/stem/oscilaciones-y-ondas/
Ideatronic. (s.f.). Módulo sensor óptico CNY70 [Video]. YouTube. https://www.youtube.com/watch?v=kA4Fn-ia7LE
NOMOS Glashütte. (s.f.). NOMOS balance spring. Recuperado el 4 de octubre de 2026, de https://nomos-glashuette.com/en/topics/glossary/nomos-balance-spring
Wikipedia. (2026, 4 de octubre). Balance wheel. https://en.wikipedia.org/wiki/Balance_wheel.
seleube (2018, 26 de agosto). Domina el Movimiento ARMÓNICO SIMPLE en 11 minutos [Video]. YouTube. https://www.youtube.com/watch?v=kk8wP0Utc6I